

MERN Stack Developer Assignment

Mini CRM Opportunity Tracker

Full-stack task to evaluate authentication, authorization, API design, React integration, and deployment readiness

1. Overview

The objective of this assignment is to build a secure full-stack MERN web application that allows users to manage a shared CRM-style sales opportunity pipeline. Instead of a simple bookmark manager, the application should feel like a small real-world internal tool used by a startup, sales team, or SME to track leads, follow-ups, and deal stages.

The system should ensure:

- Secure user authentication using JWT
- Password protection using bcrypt
- Controlled access to resources using ownership-based authorization
- Clean frontend-backend integration using React and REST APIs
- A deployed, working live application with a clear README

2. Functional Requirements

2.1 User Authentication

Registration

- Users should be able to register with:
 - Name
 - Email
 - Password
- Email must be unique
- Password must be stored using bcrypt hashing
- Basic validation should be added for required fields and email format

Login

- Users should be able to log in using email and password
- On successful login, a JWT token must be generated and returned
- Invalid credentials should return proper error messages and HTTP status codes

2.2 Opportunity Management

Create Opportunity

- Logged-in users can create opportunities with:
 - Customer / company name
 - Contact person name

- Contact email or phone
 - Requirement / need summary
 - Estimated deal value
 - Stage: New, Contacted, Qualified, Proposal Sent, Won, Lost
 - Priority: Low, Medium, High
 - Next follow-up date
 - Notes
- The creator of the opportunity must be derived from the JWT token on the backend
 - Do not accept user_id or created_by from the frontend request body

View Opportunities

- All logged-in users can view all opportunities in a shared pipeline
- Each opportunity card or row must display:
 - Customer / company name
 - Requirement summary
 - Deal value
 - Stage
 - Priority
 - Next follow-up date
 - Created by user name
 - Created date
- Users should be able to identify which opportunities belong to them

Update Opportunity

- Users can update only opportunities created by themselves
- Users must NOT be able to edit opportunities created by others
- At minimum, users should be able to update stage, priority, next follow-up date, value, and notes

Delete Opportunity

- Users can delete only opportunities created by themselves
- Users must NOT be able to delete opportunities created by others
- Ownership validation must happen in the backend, not just in the frontend UI

3. Security Requirements

- Implement JWT-based authentication
- Pass token in request headers as: Authorization: Bearer <token>
- Extract user identity from JWT in backend middleware
- Do NOT accept user_id from frontend for create, update, or delete operations
- Enforce ownership validation in backend before update and delete operations
- Use bcrypt for password hashing

- Protect all opportunity APIs so only authenticated users can access them
- Store sensitive values such as JWT secret, database URL, and environment-specific configuration in environment variables

4. Technical Requirements

Backend

- Technology: Node.js with Express.js
- Database: MongoDB using Mongoose
- Architecture: Modular structure with routes, controllers, services, models, middleware, and config
- Authentication: JWT + bcrypt
- Validation: Basic request validation using a clean approach such as middleware, Joi, Zod, express-validator, or custom validation
- API format: RESTful JSON APIs

Frontend

- Technology: React.js, preferably using Vite
- Pages: Login, Register, Opportunity Dashboard, Create / Edit Opportunity
- Use Axios or fetch for API calls
- Store JWT in localStorage, sessionStorage, or secure cookie with approximately 2 hours expiry
- Show clear loading, success, and error states
- Frontend should hide edit/delete actions for records not owned by the logged-in user, while backend must still enforce authorization

5. Suggested Project Structure

Backend Structure

```
/backend
├── src/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   ├── authController.js
│   │   └── opportunityController.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   └── errorMiddleware.js
│   ├── models/
│   │   ├── User.js
│   │   └── Opportunity.js
│   └── routes/
```

```

| | └─ authRoutes.js
| | └─ opportunityRoutes.js
| └─ services/ (optional)
| └─ utils/ (optional)
└─ server.js
└─ package.json
└─ .env.example
  
```

Frontend Structure

```

/frontend
└─ src/
  | └─ components/
  | | └─ Navbar.jsx
  | | └─ OpportunityCard.jsx
  | | └─ OpportunityForm.jsx
  | └─ context/
  | | └─ AuthContext.jsx
  | └─ pages/
  | | └─ Login.jsx
  | | └─ Register.jsx
  | | └─ Dashboard.jsx
  | └─ services/
  | | └─ api.js
  | └─ App.jsx
  └─ main.jsx
└─ package.json
└─ .env.example
  
```

6. Database Models

User Model

Field	Type	Notes
_id	ObjectId	Auto-generated MongoDB ID
name	String	Required
email	String	Required, unique
password	String	Required, bcrypt hashed
createdAt	Date	Auto-generated timestamp

updatedAt	Date	Auto-generated timestamp
-----------	------	--------------------------

Opportunity Model

Field	Type	Notes
_id	ObjectId	Auto-generated MongoDB ID
owner	ObjectId	Reference to User; set from JWT
customerName	String	Required
contactName	String	Optional
contactEmail	String	Optional
contactPhone	String	Optional
requirement	String	Required short summary
estimatedValue	Number	Optional, should be non-negative
stage	String	Enum: New, Contacted, Qualified, Proposal Sent, Won, Lost
priority	String	Enum: Low, Medium, High
nextFollowUpDate	Date	Optional
notes	String	Optional
createdAt	Date	Auto-generated timestamp
updatedAt	Date	Auto-generated timestamp

7. API Requirements

Authentication APIs

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Login and return JWT token
GET	/api/auth/me	Optional: return logged-in user profile

Opportunity APIs

Method	Endpoint	Access Rule
GET	/api/opportunities	Logged-in users can view all opportunities

POST	/api/opportunities	Logged-in users can create opportunities
GET	/api/opportunities/:id	Logged-in users can view a specific opportunity
PUT	/api/opportunities/:id	Only the owner can update
DELETE	/api/opportunities/:id	Only the owner can delete

8. UI / UX Requirements

- Login and registration forms should be clear and usable
- Dashboard should show opportunity cards or a table with stage, priority, next follow-up date, and owner
- Add filters for stage and priority if possible
- Use simple, clean styling; Tailwind CSS, CSS modules, Bootstrap, or plain CSS are all acceptable
- Show meaningful empty states, validation errors, and API error messages
- The app should be usable on desktop and reasonably usable on mobile

9. Evaluation Criteria

- Working live deployed application
- Correct implementation of JWT authentication
- Secure password handling using bcrypt
- Proper backend authorization with ownership validation
- Clean MERN architecture and modular code structure
- Good API design and REST standards
- Frontend-backend integration quality
- Error handling, form validation, and correct HTTP status codes
- Code readability, naming, folder structure, and maintainability
- README quality and ease of setup

10. Bonus / Optional Enhancements

- Pagination, search, and filtering on opportunities
- Kanban-style board by stage
- Sorting by deal value, priority, or next follow-up date
- Activity notes or follow-up history per opportunity
- Dashboard summary cards such as total pipeline value, won value, and high-priority opportunities
- Improved UI/UX with responsive design
- Centralized logging and exception handling
- Unit tests or API tests for authentication and ownership validation
- Docker setup for backend and frontend

11. Important Notes

- Do not store passwords in plain text
- Do not trust frontend data for user identity
- Do not rely only on hiding buttons in the frontend for authorization
- Ensure proper HTTP status codes are returned
- Use environment variables and include a sample .env.example file
- Code should be clean, readable, and well-structured
- Do not commit real secrets, database credentials, or JWT secret keys to GitHub

12. Submission Guidelines

Candidate must provide:

- Live application link
- GitHub source code repository link
- Test login credentials, if registration is disabled or restricted
- README file with setup and deployment details

README should include:

- Project overview
- Tech stack used
- Backend setup instructions
- Frontend setup instructions
- Environment variables required
- API details
- Deployment steps
- Known limitations or pending improvements

13. Expected Outcome

- Users can securely register and log in
- Authenticated users can create opportunities
- All authenticated users can view the shared opportunity pipeline
- Only the creator of an opportunity can edit or delete it
- Application is deployed and accessible online
- Security and architecture best practices are followed

14. Deployment Requirement (Mandatory)

- The application must be deployed and accessible via a live URL
- Both frontend and backend should be hosted
- Database should be hosted and connected to the deployed backend
- Candidate must provide live application link and GitHub code link
- Submissions without a working live link will be considered incomplete

Accepted platforms include, but are not limited to:

- Frontend: Vercel / Netlify
- Backend: Render / Railway / AWS / VPS
- Database: MongoDB Atlas or equivalent hosted MongoDB service